AuditionSense: A Modular Multi-modal AI System for Human Performance Analysis

Submitted By
Mehraeel Philip
Fatema Milad
Sama Gehad
Marwan Ahmed
Mo'men Mohie

A dissertation submitted in partial fulfillment of the requirements for the degree of
Computing & Digital Technology
in the
School Of Computing & Digital Tech
of
ESLSCA University Egypt

Graduation Projects advisor:
Dr. Ayman Atia
May 2026

Abstract

AuditionSense is a multi-modal AI-based system that analyzes human performance by combining visual, auditory, and behavioral data. The system is made up of several layers that work together to allow users to access the system easily through its interface. The primary performance analysis module centralizes the outputs from different sub-modules: emotion recognition, eye tracking, gestural recognition, pose estimation, and voice analysis. Each sub-module uses machine learning and computer vision techniques to derive useful features from video files submitted by users. Supporting utilities pre-process the data, extract features, and produce visualizations; visualizations are produced using Matplotlib. Data being processed in a defined structure (pipeline) to produce detailed reports of the user's performance metrics and behaviors. Experiments show that AuditionSense can provide accurate results from large datasets, making it applicable for use in training, evaluation and human-computer interaction (HCI). Due to its modular structure, there is potential for adding advanced models and real-time processing capabilities in the future.

Acknowledgment

We would like to extend heartfelt thanks to Dean Hazem AbdelAzim of the School of Computing for his continual encouragement and for creating an educational environment which allows for innovation and growth.
We would like to thank Dr. Magi Hossam, our Academic Director for the guidance and support she has given us throughout our entire academic career.
We cannot express enough thanks to our Supervisor Dr. Ayman Ezzat for his incredible mentorship, guidance during each stage of this project, and assistance in every way possible. His guidance and commentary greatly improved the quality of our work.
We would also like to thank Dr. Zeinab Samir and Dr. Ezzeldin Sherif for their valuable observations and helpful suggestions that helped to refine our project.
Our sincere appreciation goes to Dr. Reem ElMeligui and Mr. Wael Ehsan for the inspiring advice and positive encouragement they provided to us, which gave us the bravery to go ahead and succeed with our project.
We want to thank Adam Hassan, Laila Sherif and Nejoud Amr for being with us every step of the way and for their encouraging words and sustaining support.
In conclusion, we acknowledge all the many respected physicians, colleagues and other peers involved in the successful completion of this project via their assistance, support and positive influence.

Contents

List of Tables

List of Figures

1 Project Proposal

1.1 Introduction

1.1.1 Background

Acting is closely connected to the way individuals express themselves through body language, facial expressions, and physical postures. Non-verbal cues often reveal emotions and mental states that words may not fully express. This project explores the analysis of human posture, gestures, and emotional states to classify acting performance. Using computer vision and behavior analysis techniques, the system captures and interprets body language and emotions to offer insights. This is a modular system that analyze short audition clips (5 minutes maximum), extracts multi-modal features (gaze, emotional affect through face, voice, pose/gesture), segments the actions in the streams, scores/annotates the performer's performances, and presents director-facing reports through an interactive user interface. The audience consists of filmmakers (to consume the report) and actors (the subjects of the analysis).

In traditional audition methods within the entertainment industry, subjective human judgment plays an important role, but there can be many variables that affect this judgment, such as fatigue, personal preferences, and subconscious biases. This project aims to change that by creating a complete web-based solution for automating the analysis of performers in terms of their emotion detection via backend systems, tracking their gaze stability, and producing detailed reports.

This system is designed to connect artistic performances to quantitative evaluations and provide casting directors, talent agents, and production companies with an extremely reliable resource for assessing audition recordings with higher levels of accuracy and consistency than was previously possible.

1.1.2 Project Motivation

Utilizing cutting-edge artificial intelligence and computer vision technology, the AI-Powered Audition Performance Analysis System is a revolutionary innovation in assessing talent for the entertainment industry. This AI-assisted tool takes the guesswork out of assessing actor audition performances by providing objective, evidence-based feedback from audition performances to eliminate personal bias when making casting choices.

By developing a system that can automatically detect human emotions, postures, and gestures using computer vision and artificial intelligence, the aim is to enhance casting auditions. This project will allow directors to observe actors' performance through recorded sessions, helping them track progress objectively. Ultimately, our passion lies in making **behavioral analysis more data-driven, accessible, and supportive** — not replacing human judgment but amplifying it with intelligent insights

Directors assess performer's first by evaluating their eyes and emotions, then their voice, and finally their posture/gestures; a five minute audition can benefit from feedback that is concise, descriptive, and rated rather than binary direct classification (suggested practice from a professional director). Automating objective multi-modal measurements can increase the reliability of consistent feedback during auditions and accelerate the triage process during the auditions.

1.2 Problem Statement

Directors face significant challenges in accurately assessing actors' emotions and behavioral states due to the subjective nature of human observation and the limited availability of quantitative data. Nonverbal cues such as facial expressions, posture, and gestures play a critical role in performance results, yet they are often overlooked or misinterpreted, especially during long sessions.

Directors need instant feedback and evaluation on an objective basis when auditioning talent quickly. Manually reviewing audition footage takes too long and is subjectively evaluated. Current tools do not allow for an integrated multi-modal pipeline tool that matches the order of evaluation preferred by directors (i.e. visual aspects/affective aspects, voice elements, physical elements) and report these aspects of an audition via a short written summary, which can be rated against a score.

Furthermore, there is a lack of automated systems capable of integrating multiple behavioral indicators—facial emotion, posture, and gesture to form a comprehensive audition report. Therefore, there is an urgent need for an AI-driven behavioral analysis system that can detect, analyze, and interpret emotional and postural patterns to assist directors in saving time and effort in a more objective, continuous, and insightful manner.

1.3 Project Objectives

1. Accurately estimate eye gaze and facial emotions aligned with scene context.
2. Extract voice features (pitch, intensity, tempo) and transcribe speech.
3. Detect and classify gestures/postures; measure gesture/template matching accuracy using $1 recognizer.
4. Segment long stream videos into discrete actions (10 actions) and report per-action detection accuracy.
5. Produce concise director-facing reports: short description (1–3 sentences) + per-dimension numeric scores (0–100) + visuals.
6. Provide reproducible pipelines and user-friendly UI for uploads and report viewing.

1.4 Proposed Solution

Multi-modal Analysis: Rather than just using one source of input, this system gathers separate types of behavioral signals at once, increasing the reliability and accuracy of the analysis.
Automated Processing: This system has automated its entire process from acquiring data through generating reports, which leads to less human involvement and contributes to less variance in interpretation.
Real-Time and Structured Workflow: The use of layers in the architecture of the system allows fast processing of information and modular communication among all of the system's parts.
Scalable and Extensible Design: Because of the modularity of the system, it is possible to add new AI modules without affecting current functions.
Increase in Efficiency for Users: The implementation of a web interface for use by the end-users via flask makes the uploading of files and retrieval of results very easy for end-users.

1.5 Project Scope

The scope of this project covers the **design, development, and evaluation of an AI-based behavioral analysis system** that can detect, classify, and interpret human emotions, gestures, and postures through computer vision and deep learning techniques. The system will process uploaded video files, making it suitable for real-time use in casting settings or environments. The focus will be on implementing and integrating core modules:

- facial emotion recognition
- posture and gesture analysis
- behavior-based interpretation
- recommendation engine that provides insights and feedback based on detected patterns.

A user-friendly graphical user interface (GUI) will be developed to allow directors to interact with the system intuitively — viewing analytics, reports, and real-time monitoring results. The scope also includes validating the system's performance through controlled testing using custom recordings. However, the project does not aim to replace professional classification; instead, it seeks to augment auditions analysis and enhance personal understanding of actors behavior. Future extensions such as multi-modal inputs (e.g., voice data) and large-scale deployment are considered beyond the current scope but may be explored in subsequent phases.

Features to be delivered:

- Processing pipelines for: eye/gaze tracking, facial landmark and emotion analysis, voice recognition, body pose and gesture recognition, and temporal action segmentation for streams.

- Gesture recognition using the One Dollar Unistroke Recognizer for template based action matching and measurement across videos.
- Performance report generator (short textual description + numeric scores).
- A simple web/data-app UI for uploading videos and viewing reports.
- ML model experiments and evaluation managed for prototyping and model comparisons

1.6 Feasibility Analysis

- Technical Feasibility: The team will use Python, OpenCV, MediaPipe, and emotion recognition libraries. Hardware requirements are minimal, limited to smart device connected to the internet.
- Operational Feasibility: The system can be implemented within the academic time-frame using open-source tools.
- Economic Feasibility: All tools are open-source; no extra cost is needed.
- Schedule Feasibility: The full functioning project attached to the final document all related phases.

1.7 Expected Outcomes

- A **functional prototype** capable of detecting and classifying emotions, gestures, and postures from uploaded video inputs.
- A **graphical user interface (GUI)** allowing users to interact with the system intuitively.
- **Real-time emotion and posture analytics**, visualized through scoring widgets for better understanding.
- **Analysis Report** to have full detailed evaluation of the performance
- A **recommendation engine** that interprets behaviors (e.g., "frequent eye gazing signifies stress or anxiety") and provides helpful insights.
- A **proof of concept** demonstrating the integration of computer vision, AI-based emotion detection, and interpretation in a unified system.

2 System Requirements Specification (SRS)

2.1 System Overview

Key Components:

- Data Ingestion: either file upload, or capturing via the webcam (video + audio).
- Data Pre-processing: extracting frames, extracting audio, detecting face/pose bounding boxes, synchronizing timestamps.
- Per-modality analyzers:
  - Eye/Gaze Estimator (MediaPipe Iris/FaceMesh + calibration).
  - Facial Emotion Detector (FER / DeepFace emotion module).
  - Voice Analyzer (using speech-to-text + prosody from python libraries).
  - Pose and Gesture Pipeline (using OpenCV + MediaPipe Pose / dlib landmarks + 1 Dollar recognizer for gestures).
- Action Segmentation Module: temporal segmentation of streaming videos (using temporal action segmentation techniques; can use frame features fed in to TCN/transformer-based models)
- Scoring and Interpretation: rule aggregator + ML-based model (e.g., PyCaret for fast experimentation).
- UI: Flask web app to show scores, short textual summaries, and PDF reports.
- Storage and Artifacts: local file structure to store video samples, extracted features, models, reports, and evaluation

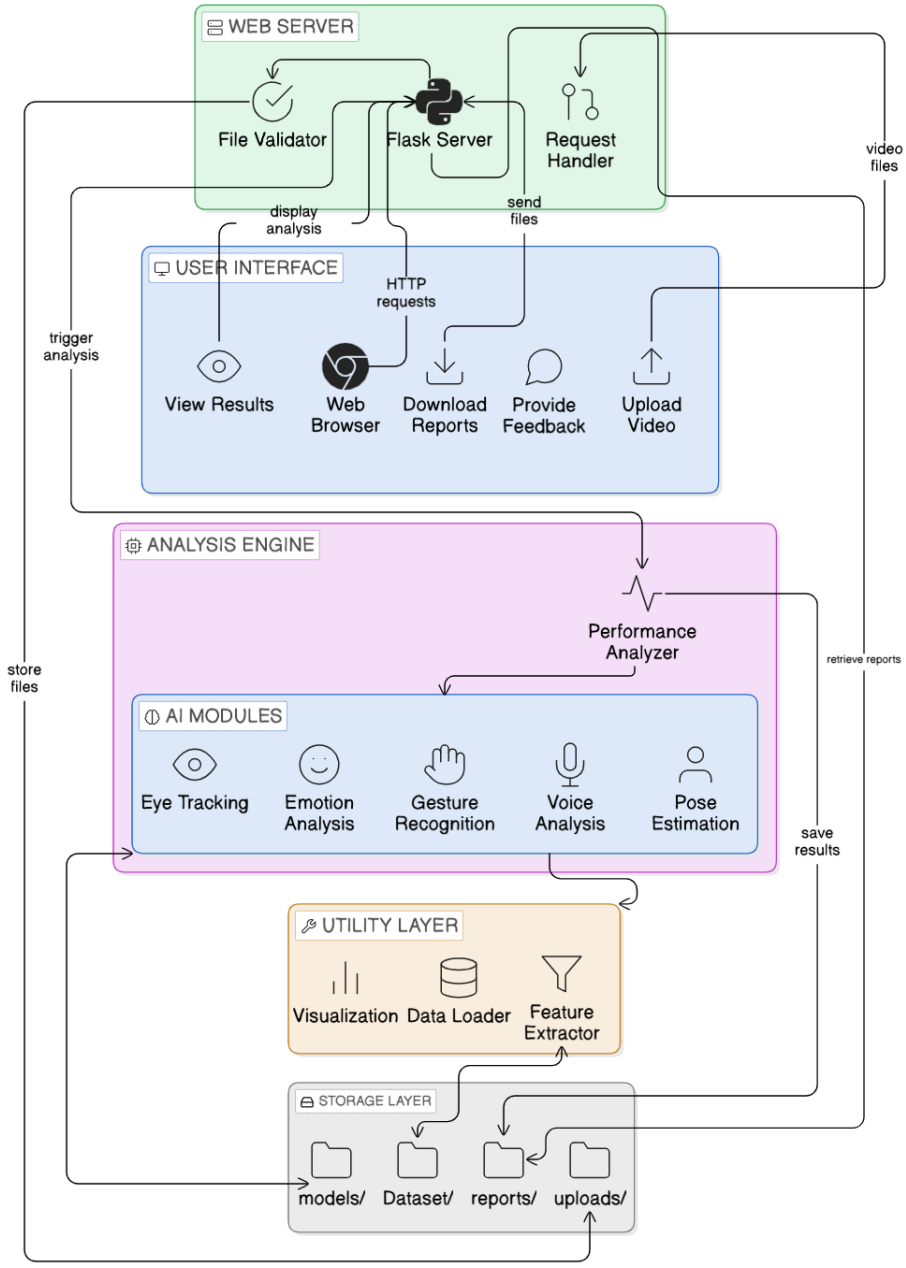

Figure 1: System Overview Diagram

2.2 Functional Requirements

FR-01: Video Upload

- Description: System shall accept video files (mp4, avi, mov) of 1-minute to 5-minute audition takes.
- Input: video file and actor data (name, age, age group)
- Output: stored raw video, extracted audio (wav), metadata record.
- Acceptance: uploaded video and able to start analysis

FR-02: Facial Landmark & Eye Tracking Module

- Description: Detect face landmarks and iris/eye region per frame, compute gaze direction & relative eye movement metrics.
- Tech: MediaPipe FaceMesh/Iris + OpenCV preprocessing.
- Outputs: per-frame landmark coordinates, gaze vector, gaze heatmap (aggregated).
- Acceptance: landmarks detected for 80% of frames with visible face; gaze displays clustering of fixations.

FR-03: Facial Emotion Estimation

- Description: Per-frame emotion probabilities (basic Ekman-like categories: joy, sadness, anger, fear, surprise, disgust, neutral); aggregate to per-scene/emotion profile.
- Tech: FER library / DeepFace emotion models (both available in Python).
- Outputs: time-series of emotion probabilities; scene-aware summary (dominant emotion vs expected scene emotion).
- Acceptance: model produces plausible dominant emotion aligned with visual inspection; API returns probabilities per frame.

FR-04: Pose / Gesture Detection & $1 Recognizer Measurement

- Description: Extract 2D/3D pose keypoints, segment gestures, and match to template gestures using $1 Recognizer; compute match score and count correct detections
- Tech: MediaPipe Pose / OpenPose-like keypoints; implement $1 recognizer for gesture templates (unistroke) for simple gesture shapes; measure hits per stream.
- Outputs: per-gesture score, boolean correct/incorrect per template, confusion matrix
- Acceptance: reported metric: number of correctly detected actions per-actor classification accuracy.

FR-05: Report Generation

- Description: For each audition, produce a brief textual description (1–3 sentences) and numeric scores for: eye engagement, emotional fidelity, vocal delivery, gesture/posture, overall.
- Mechanism: weighted aggregation of normalized metrics (weights configurable; director preference: eye/emotion highest priority).
- Outputs: textual summary + JSON score report + visualization widgets (individual score).

- Acceptance: description must be 3 sentences and include at least one concrete observation (e.g., “sustained gaze at camera 45% of audition; dominant emotion: sadness”).

FR-06: UI Report Viewer

- Description: Flask app to upload files and view/download reports; interactive timeline with overlays (emotion probabilities, gaze fixations, detected gestures).
- Outputs: visualization, download PDF locally.
- Acceptance: report view loads within ˜5 seconds for a 1-minute video on mid-range laptop.

FR-07: Evaluation & Experiment Logging

- Description: Save experiment runs, model versions, evaluation metrics, and confusion matrices for reproducibility. PyCaret can be used to compare models for classification subtasks.
- Outputs: evaluation logs (JSON), saved model artifacts.

2.3 Non-functional Requirements

NFR-01: Accuracy & Reliability

- Target: facial landmark detection 80% frames; emotion classification baseline 60% (subject to dataset and human variability); gesture matching accuracy to be reported per-experiment (no fixed guarantee).
- Rationale: emotion and gaze are hard; accuracy depends heavily on dataset quality and lighting.
- Metric to measure is the compatibility of actual input with the detected output.

NFR-02: Performance

- Processing models: (a) Real-time (approx. streaming inference) for preview with subsampled models; (b) Offline batch (full analysis, slower).
- Target offline processing: 1 minute of video processed in 10 minutes on a laptop with GPU if available (empirical tuning required).
- Metrics used to measure performance: time taken to detect, processing capacity, speed, accuracy, f1 score, precision and recall.

NFR-03: Usability

- UI must be simple: upload → analyze → short report + visuals.
- Director-oriented language; avoid technical jargon.
- Time taken to learn how to use the system.
- Metrics to measure can be SUS

NFR-04: Portability

- Runs on Windows (primary dev target) and Linux, most operating systems.
- Use Python 3.9+ virtual environment.

NFR-05: Reproducibility & Traceability

- All results must include model versions, timestamp, and seed.

NFR-06: Privacy

- Store raw videos locally and allow explicit deletion.
- No cloud storage by default.

2.4 System Diagrams

2.4.1 Use Case Scenarios

1. **UC-01 Upload Video:**
   (a) Actor: Director
   (b) Precondition:
      i. Director is logged into the system.
      ii. Director has a video file ready for upload.
   (c) Main Success Flow:
      i. Director opens the upload page.
      ii. Director enters actor/audition information.
      iii. Director selects a video file.
      iv. System validates the uploaded file type.
      v. System stores the video in storage/database.
      vi. System extracts audio and metadata.
      vii. System initializes the analysis pipeline.
      viii. System displays upload success message.

(d) Alternative Flow:

i. If uploaded file format is invalid:

- system displays an error message.
- director uploads another valid file.

(e) Post-condition: Video is successfully uploaded and prepared for analysis.

2. **UC-02 Analyze Performance:**

(a) Actor: Director

(b) Precondition:

i. A valid audition video is uploaded.

ii. Analysis modules are available.

(c) Main Success Flow:

i. Director selects “Analyse Performance”.

ii. System validates uploaded media type.

iii. System extracts frames and audio.

iv. System dispatches data to AI modules.

v. AI modules preprocess the data.

vi. System performs emotion, eye, gesture, pose, and voice analysis.

vii. System computes metrics and confidence scores.

viii. System combines analysis results.

ix. System generates performance insights and scores.

x. System stores analysis results and metadata.

xi. Director views analysis results.

xii. Director downloads or saves the report.

(d) Alternative Flow:

i. If uploaded media is invalid:

- system displays an error message.
- director uploads another valid media file.

(e) Post-condition: Performance analysis results are successfully generated and stored.

3. **UC-03 Emotion Detection:**

(a) Actor: Director

(b) Precondition: Director uploads a valid image or video.

(c) Main Success Flow:

i. Director starts emotion detection.
ii. Director uploads image or video.
iii. System validates uploaded media type.
iv. System extracts frames if media is video.
v. System detects emotions.
vi. System analyzes emotion scores and confidence levels.
vii. System generates emotion detection result.
viii. System stores result in storage/database.
ix. Director views the detection result.
x. Director downloads or saves the result.

(d) Alternative Flow:

i. If uploaded media is invalid:
- system displays an error message.
- director uploads another valid file.

ii. If no face is detected:
- system displays warning message.
- emotion analysis cannot continue.

(e) Post-condition: Emotion detection result is successfully generated and saved.

4. **UC-04 Generate Report:**

(a) Actor: Director

(b) Precondition:

i. Performance analysis is completed.
ii. Analysis data exists in the system.

(c) Main Success Flow:

i. Director selects “Start Analysis”.
ii. System retrieves analysis data.
iii. System organizes scores, metrics, and insights.
iv. System creates report visuals and summaries.
v. System generates final report document.
vi. System stores generated report.
vii. System displays success message.

(d) Alternative Flow:

i. If analysis data is unavailable:

- system displays an error message.
- report generation process stops.

(e) Post-condition: Performance report is successfully generated and stored.

5. **UC-05 View Report:**

(a) Actor: Director

(b) Precondition: Generated report exists in storage.

(c) Main Success Flow:

i. Director opens reports page.
ii. Director selects actor/audition report.
iii. Director clicks “View Full Report”.
iv. System retrieves report data and visuals from storage.
v. System loads report content.
vi. System displays scores, insights, and visual analysis.
vii. Director navigates through report pages.
viii. Director closes report page.

(d) Alternative Flow:

i. If report is unavailable:
- system displays “Report Not Found” message.

(e) Post-condition: Director successfully views the generated report.

6. **UC-06 Download / Save Report:**

(a) Actor: Director

(b) Precondition: Report has already been generated.

(c) Main Success Flow:

i. Director opens generated report.
ii. Director selects Download or Save option.
iii. System prepares report file.
iv. System downloads/saves report in selected format.
v. System displays confirmation message.

(d) Alternative Flow:

i. If download/save process fails:
- system displays an error message.
- director retries the process.

(e) Post-condition: Report is successfully downloaded or saved.

2.4.2 Use Case Diagram

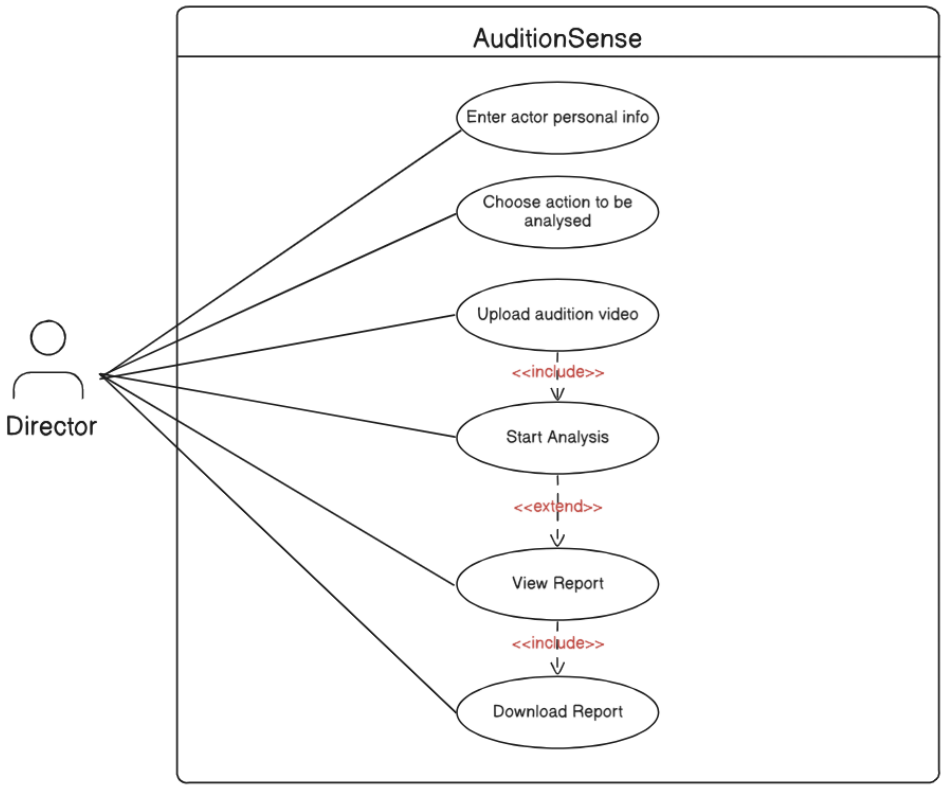

Figure 2: Use Case Diagram

2.4.3 Activity Diagram

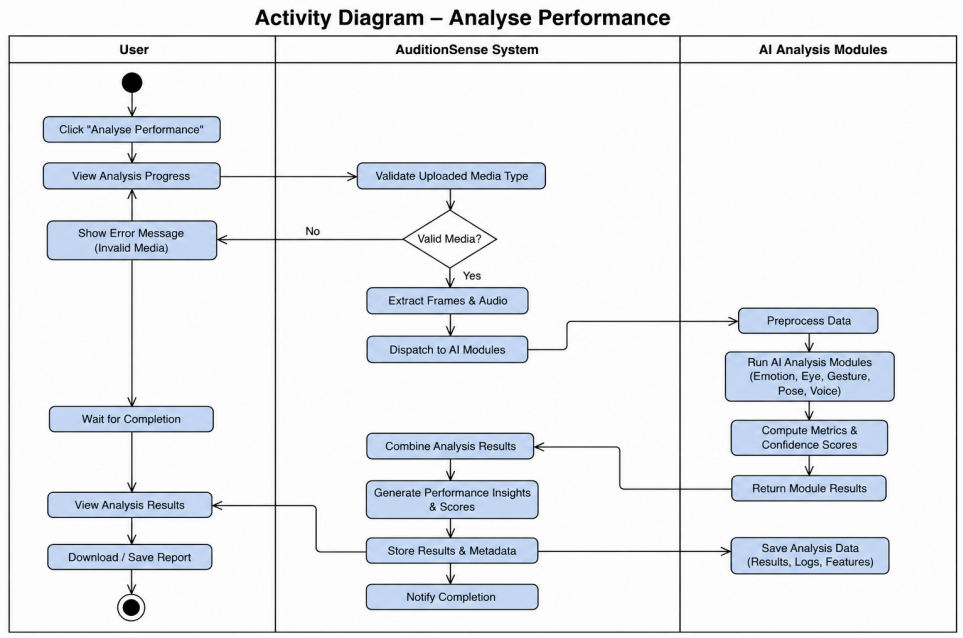

Figure 3: Analyze Performance Activity Diagram

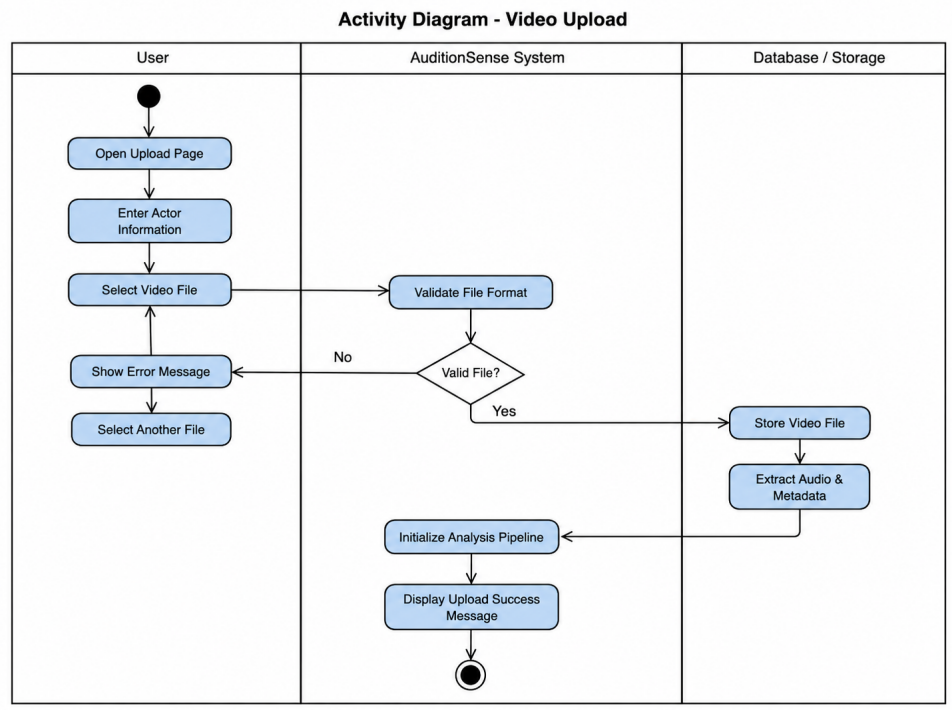

Figure 4: Video Upload Activity Diagram

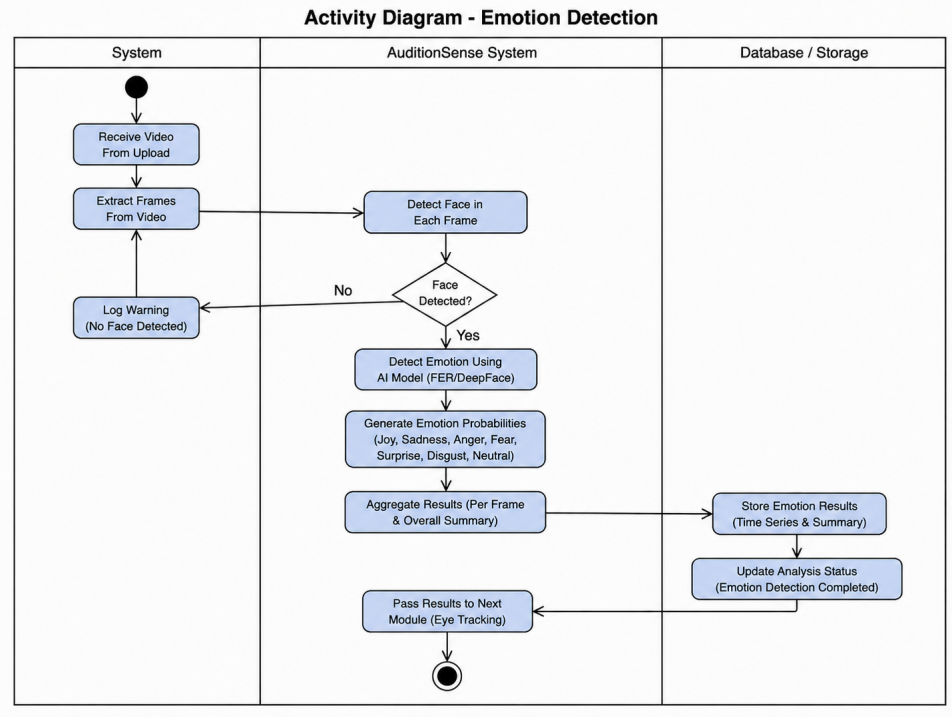

Figure 5: Emotion Detection Activity Diagram

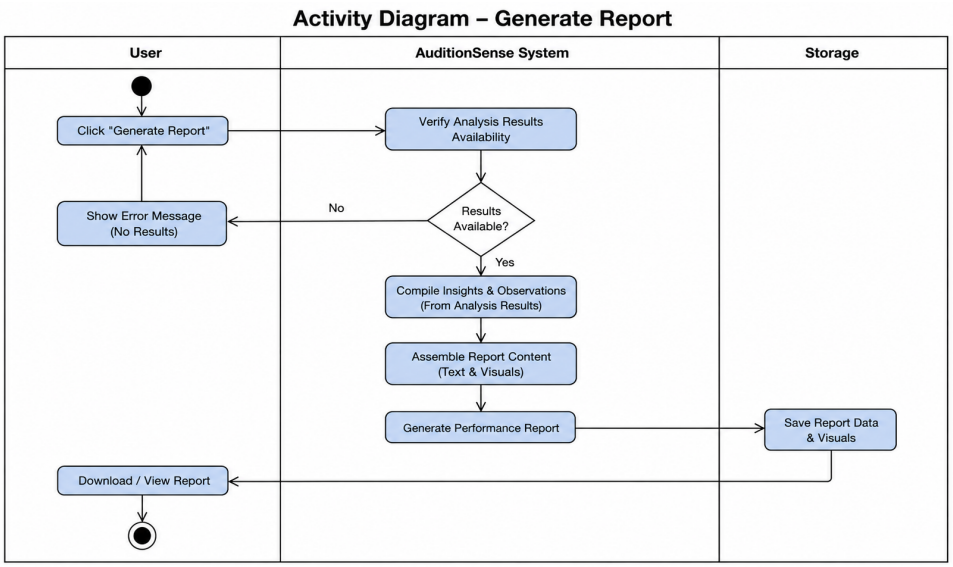

Figure 6: Generate Report Activity Diagram

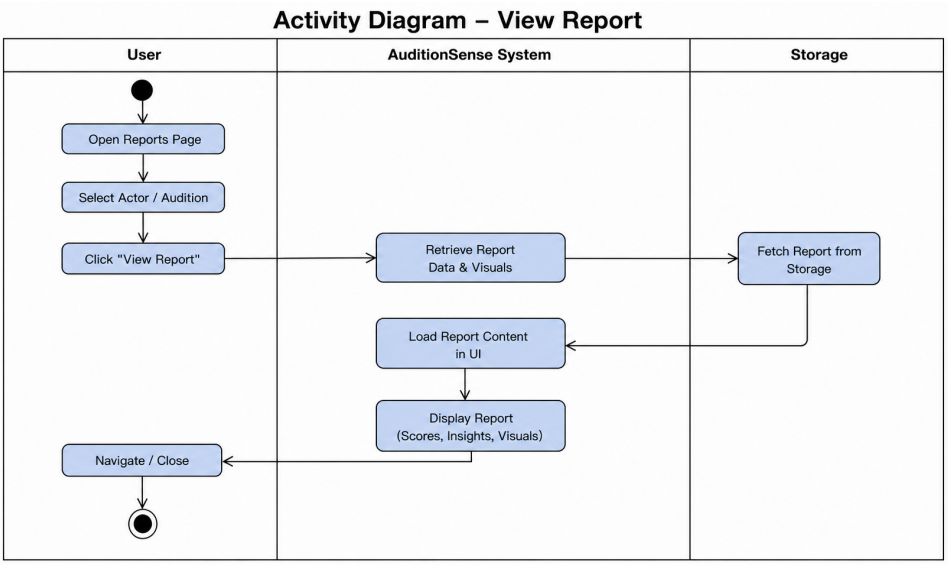

Figure 7: View Report Activity Diagram

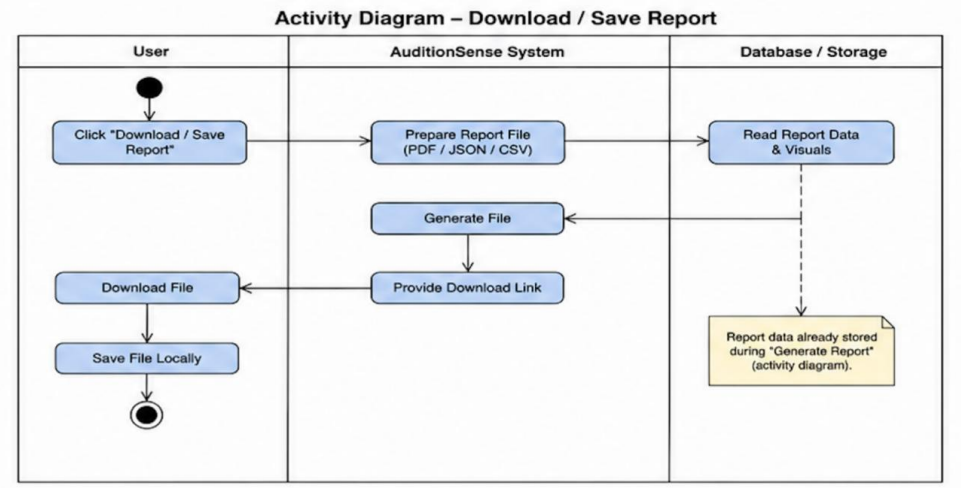

Figure 8: Download Report Activity Diagram

2.4.4 Sequence Diagram

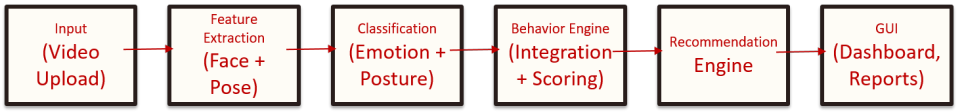

Figure 9: Sequence Diagram

2.5 Quality Assurance and Evaluation Plan

Unit tests for pre-processing, landmark extraction, and simple end-to-end benchmarks.

- Split dataset: per-actor stratification ensuring an appropriate size number of actions and actors are represented.
- Metrics:
  - Emotion: frame-level accuracy with F1 per class
  - Gaze detection: precision/recall for fixation detection (if ground truth is available)
  - Gesture detection: recognition accuracy and 1 dollar recognizer match rate
  - Action segmentation: frame-wise accuracy, edit distance, mIoU (standard TAS metrics)
- Human-in-the-loop: director ratings where it makes sense as gold labels for ”useful summary” evaluation.

3 System Design Document (SDD)

3.1 Architectural Design

3.1.1 Architecture Overview

The system is based on an Event-Driven Architecture that takes advantage of a client-server architecture to provide communication between system components via Events instead of Calls; this allows for loose coupling, scalability, and asynchronous processing.

The architecture consists of four logical layers:

1. Presentation Layer.
2. Event Processing and Orchestration.
3. Analysis and AI Processing and
4. Reporting and Storage.

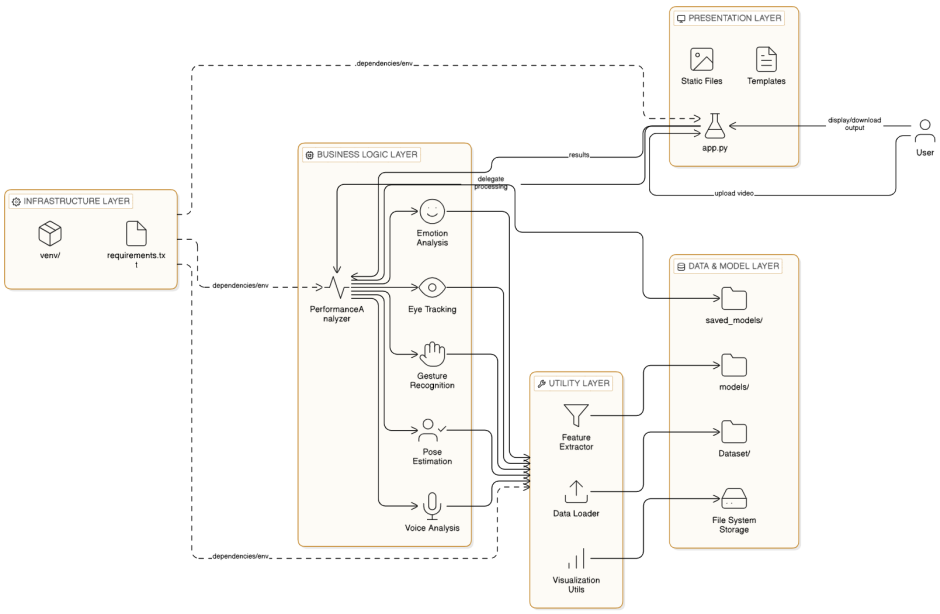

Figure 10: Layered Architecture Diagram

Whenever a major event takes place, an Event is created and used to trigger further actions by one or more of the components.

There are many advantages to using Event-Driven Architecture for this project:

1. Since both recording and uploading of auditions are asynchronous, they can be performed concurrently.

2. The amount of computation required to detect and analyze a recording can be high.

3. Several different AI modules operate really independently of each other, therefore it is easy to scale the system by adding new modules (emotion detectors, voice analyzers, etc.) at a later time.

4. Using this approach to construct the system improves scalability, fault tolerance, and modularity.

3.1.2 Event Flow Description

The system operates based on the following events:

1. Actor Information Submitted Event **Event:** `ActorProfileSubmitted`

Triggered when: The user enters actor information (name, age, role, audition ID).
Produced by: Client Application (UI)
Consumed by:

- Backend Orchestrator
- Database Service

Purpose:

- Validate and store actor metadata
- Initialize an audition session

2. Audition Recorded or Uploaded Event **Event:** `AuditionMediaReceived`

Triggered when:

- The user records an audition via camera
- OR uploads a pre-recorded video/audio file

Produced by: Client Application
Consumed by:

- Media Storage Service
- Event Orchestrator

Purpose:

- Store raw audition media
- Trigger analysis pipeline

3. Detection Event **Event:** `AuditionDetectionStarted`

Triggered when: Audition media becomes available.

Produced by: Event Orchestrator

Consumed by:

- Computer Vision Module
- Audio Processing Module

Detection Tasks Include:

- Face detection
- Hand or body movement detection
- Speech presence detection

4. Analysis Event **Event:** `PerformanceAnalysisCompleted`

Triggered when: Detection results are ready.

Produced by: AI Analysis Services

Consumed by:

- Scoring Engine
- Report Generator

Analysis Includes:

- Gesture consistency
- Facial expression dynamics
- Voice clarity and confidence
- Temporal performance patterns

5. Report Generation Event **Event:** `PerformanceReportGenerated`

Triggered when: Analysis and scoring are completed.

Produced by: Report Generator Service

Consumed by:

- Client Application
- Database

Output:
A structured performance report containing:

- Actor profile
- Performance metrics
- Overall score
- Observational insights

3.1.3 Technology Stack

The system uses a hybrid AI and application stack, chosen for performance, scalability, and research suitability.

Frontend / Client

- **Python Flask (Custom UI)** – User interaction, media capture, visualization
- **OpenCV** – Real-time camera handling

Backend & Orchestration

- **Python** – Core backend logic
- **Event Queue (in-process or message-based)** – Event dispatching and coordination
- **Multithreading / Async Processing** – Parallel detection and analysis

AI & Analysis Layer

- **MediaPipe** – Hand and body landmark detection
- **OpenCV** – Image processing
- **NumPy & SciPy** – Feature extraction and numerical analysis
- **Machine Learning Models (custom / pretrained)** – Performance scoring

Data & Storage

- **Local File Storage** – Video storage, actor metadata and reports
- **CSV / JSON** – Structured result storage

Reporting

- **Python Report Generator** – Performance summaries
- **PDF / Structured Output** – Final audition report

3.2 Component Design

3.2.1 Overview

AuditionSense is made up of a variety of modules that can be stacked together to make it easy to grow, maintain and analyze multimedia files effectively. AuditionSense is made up of many separate but connected modules—each module has been designed to perform its function independently and in conjunction with other modules as part of the overall processing workflow.

The requirements for the design are:

- Separation of Responsibility
- Modular Components for Re-usability
- Efficient Data Processing
- Testing and Maintenance Simplification

3.2.2 Main Components and Sub-modules

1. Web Application Component
   (a) ‘web_application/‘
   The purpose of this component is to provide an interface through which users can interact with and submit requests to the analysis engine via the web (using the Flask framework).
   (b) Sub-modules
      i. Request Handler: Accepts user requests and file uploads
      ii. File Management: Manages uploaded files and produces reports
      iii. Response Renderer: Generates, displays, and produces download links for the results of AI analyses.
   (c) Inputs Multimedia files uploaded by users.
   (d) Outputs Analysis requests and results returned to users.
2. Performance Analysis Component

(a) ‘analysis/performance_analyzer.py‘
This component serves as the main controller that coordinates all of the AI analysis components and produces combined outputs which will be used to provide a final evaluation.

(b) Inputs Preprocessed multimedia data.

(c) Outputs Aggregated performance analysis results.

3. AI Analysis Modules

(a) ‘analysis/‘
The AI analysis modules perform independent analysis of various aspects of user behavior.

(b) Sub-modules

i. Emotion Analysis: Emotion detection through facial expression
ii. Eye Tracking: Tracking of eye movements and focusing of attention
iii. Gesture Recognition: Recognition of hand and body gestures
iv. Pose Estimation: Evaluation of user posture and motion
v. Voice Analysis: Evaluation of voice characteristics

(c) Inputs Multimedia files that contain video and audio data.

(d) Outputs User behavioral metrics and insight from analysis.

4. Utility Component

(a) ‘utils/‘
The purpose of the Utility Component is to provide reusable functionality to process data.

(b) Sub-modules

i. Data Loader: Loads and pre-processing multimedia data
ii. Feature Extractor: Extracts features necessary for producing analysis
iii. Visualization Module: Creates charts using Matplotlib.

5. Data + Storage Component

(a) ‘Dataset/‘, ‘models/‘, ‘saved_models/‘, ‘uploads/‘, ‘reports/‘
The purpose of the Data + Storage Component is to store:

i. Data Sets
ii. Trained AI Models
iii. Uploaded Files
iv. Generate Reports

3.2.3 Component Interactions

A typical interaction will involve the following:

- User will upload multimedia inputs to a web application, which will then validate the data before storing it.
- PerformanceAnalyzer will dispatch work to the AI modules to process the uploaded multimedia data independently.
- Utility modules will be used to assist in extracting features from the data and producing a visual representation of the data.
- The results from each AI module will be combined to create a final report that will be sent back to the user.

3.3 User Interface Design

The User Interface (UI) guide users through an easy, guided and intuitive way when submitting auditions and reviewing performance evaluations. The UI is focused on clarity and ease of use while keeping user cognitive load to a minimum with a step-by-step workflow approach, allowing users to successfully complete their audition without requiring technical capabilities. The interface is also compliant with the system's event-driven architecture in that it utilizes user activity (e.g., form submission, recording, uploading, view report) to initiate system events.

3.3.1 Screen Prototypes or Wireframes

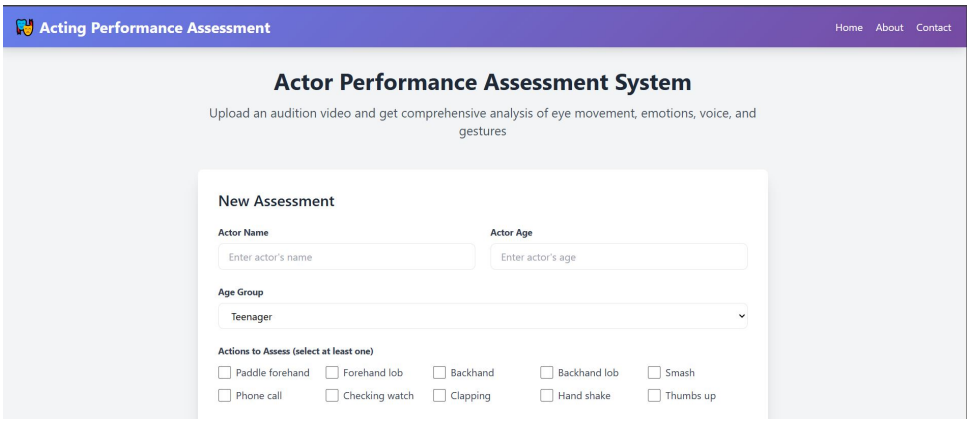

Figure 11: Home Page User Interface

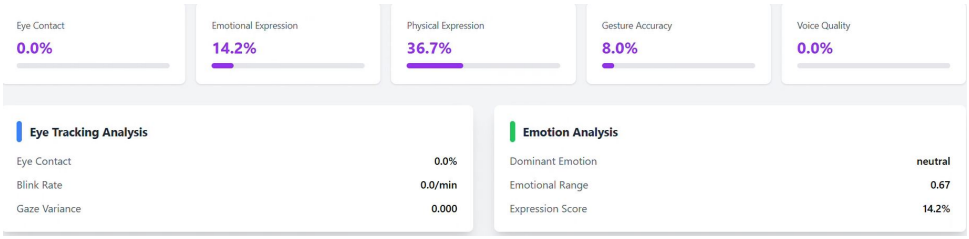

Figure 12: Sample of Scoring Widgets

3.3.2 User Interaction Flow

1. Launch Application User accesses the main screen.
2. Enter Actor Information
   - User fills in personal and audition details.
   - Submission triggers metadata storage.
3. Record or Upload Audition
   - User chooses between live recording or file upload.
   - Media is previewed before submission.
4. System Processing
   - UI displays analysis progress.
   - User input is temporarily disabled.
5. View Performance Report
   - User receives a structured evaluation report.
   - Option to download or review results.

Aspect	Description
Navigation Style	Sequential, step-based
User Control	Limited to prevent invalid states
Feedback	Visual and textual status updates
Error Handling	Validation and confirmation prompts

Table 1: Interaction Flow Characteristics

3.3.3 UI Design Justification

The interface design follows established Human–Computer Interaction (HCI) principles:

- Consistency: Uniform layout and controls across screens
- Visibility of system status: Clear processing indicators
- Error prevention: Validation before event triggering
- Learnability: Minimal instructions and intuitive controls

This makes the UI suitable for first-time users, including actors with no technical background.

3.4 External Interface

This section contains guidelines for the interaction between external software components, libraries, and services and the system. The project does not depend upon any web-based services from third parties; rather, it incorporates the use of third-party frameworks and system-level interfaces required by media processing and AI analytics.

Media Processing and Computer Vision APIs

Description:
The system uses external AI frameworks for feature detection and landmark extraction.
APIs / Libraries:

- MediaPipe (face, hand, and pose detection)
- OpenCV (image pre-processing and transformations)
- Operating system media drivers

Data Exchanged:

- Input: Video frames
- Output: Landmark coordinates and detection results

Purpose: Extract low-level features for performance evaluation
Interaction Type: Library-level API integration

Data Storage Interface

Description:
The system interacts with local storage and lightweight systems.
Technologies: File system (media files and reports)
Data Exchanged:

- Actor profiles
- Audition metadata
- Analysis results
- Generated reports

Purpose:

- Persistent storage of system data
- Enable retrieval for reporting and evaluation

Reporting and Export Interface

Description:
The system exports performance reports for user access and evaluation.
Formats Supported:

- PDF
- CSV
- JSON

Purpose:

- Allow users and evaluators to download and review results
- Support academic documentation and assessment

3.5 Traceability Matrix

Requirement ID	Architecture Component	Module	Data Entity	UI Screen
FR1	Client–Server	UI Module	Actor	Actor Info Screen
FR2	Event-Driven	Media Management	Audition	Recording / Upload Screen
FR3	Event-Driven	Detection Module	Detection Data	Processing Screen
FR4	Event-Driven	Analysis & Scoring	Analysis Results	Processing Screen
FR5	Client–Server	Report Generator	Report	Report Screen
FR6	Client–Server	UI Module	Report	Report Screen

Table 2: Traceability Matrix

4 System Implementation and Methodology

The project code is organized to specify each functionality in a clear way to ease debugging and developing (OOP). Refinements in branches including ui, reliability and feedback was carried out to improve performance generally executing the functional requirements best approach possible. Design was enhanced based on advices given by doctors and experienced directors.

Implementation - System Overflow

1. Accepts video input (recorded)
2. Decomposes video into frames for processing
3. Frames analyzed using computer vision modules
4. MediaPipe extracts 2D body landmarks (pose, hands, face)
5. Landmarks are normalized and structured
6. Pre-processing includes resizing and noise reduction
7. Processed data sent to gesture recognition stage

4.1 Functional Coverage Mapping

FR-01-02	Video Capture & Audio/Frame Sync — Data Ingestion & Preprocessing	/upload route request.form.getlist() cv2.VideoCapture()	Accepts raw 1–5 minute audition uploads, extracts 16 kHz audio, and maps synchronized timestamps.
FR-03-07	Facial, Vocal & Posture Analysis — Per-Modality Analyzers	mp.solutions.pose	Executes localized tracking for eye gaze, Ekman-like facial emotions, speech prosody, and gesture matching via the $1 Recognizer.
FR-07	Continuous Stream Slicing — Temporal Action Segmentation	recognize() analyze_video()	Segments long video streams into 10 discrete actions, assigning per-segment labels and confidence scores.
FR-08-09	Director-Facing Reporting & UI — Scoring Engine & Flask UI	chart functions JSON Report VisualizationUtils	Aggregates extracted metrics into numeric scores and 1-3 sentence text summaries, visualized on an interactive web dashboard.
FR-10	Experiment & Model Tracking — Evaluation Logging	save_results()	Saves evaluation logs, model versions, and confusion matrices to ensure complete reproducibility.

Table 3: Functional Coverage Mapping

4.2 Code Structure

4.2.1 System Overview of Code Format

- **Code Format:** The system is primarily written in Python, utilizing popular machine learning and web frameworks.
- **Core Libraries:** It relies heavily on Flask for the web interface, OpenCV and MediaPipe

for computer vision, DeepFace for emotion detection, and TensorFlow/Keras for deep learning.

- **System Use:** It is built as an "Acting Performance Assessment System" to provide comprehensive, multi-modal feedback on audition performances.

4.2.2 Architectural Structure

- Web Application (web_app/app.py): Acts as the user interface and API backend. It manages video uploads, routes users to reports, and visualizes actor comparisons using matplotlib radar and bar charts.
- Analysis Layer (analysis/emotion_analysis.py): Contains specific modules like the EmotionAnalyzer that extract frame-by-frame data (e.g., analyzing angry, disgust, fear, happy, sad, surprise, and neutral expressions).
- Machine Learning Layer (models/model_trainer.py): Manages an array of different model architectures. It is responsible for initializing, training, comparing, and saving the models.

4.2.3 Machine Learning Pipeline & Data Flow

- **Multi-Model Implementation:** The code structurally supports 8 distinct machine learning models: CNN-LSTM, LSTM, BiLSTM, GRU, RNN, CNN-RNN, Keras DNN, and PyCaret (Classical ML).
- **Data Reshaping:** Sequence models are structured to automatically reshape flat input data (e.g., transforming features into a (1, features) sequence shape to simulate time-steps).
- **Ensemble Operation:** The pipeline includes a mechanism to aggregate the predictions from all valid models using a voting ensemble and weighted probabilities to generate a final result.

4.2.4 Core Modules

- Unified Feature Extraction Module: Processes raw video and outputs a combined static/temporal data template for training.
- Single-Input Inference Engine: Houses 8 sequence-modeling architectures (CNN, RNN, LSTM variants) currently configured to process the unified data format.
- Diagnostic & Reporting Module: Generates the performance matrices (Precision, Recall, F1) that highlighted the limitations of the unified input approach.

4.3 Architecture VS Implementation Alignment

4.3.1 Planned Architecture

The system was designed as a multi-stream architecture:

1. Flow:
   Input → Feature Extraction → Parallel Streams → Fusion Level →
   Final Prediction → Visualization
2. Feature Extraction**:** Uses MediaPipe to extract pose landmarks
3. Spatial Stream (CNN): Processes posture and gesture (frame-level features).
4. Temporal Stream (RNN): Processes motion across time (sequence-based features).
5. Application Layer: Displays results and visualizations.

4.3.2 Mapping Architecture to Code Modules

- Data Layer: dataset/, analysis/, test_dataset.py, test_data_loading.py
- Feature Extraction Layer: utils/mediapipe_extractor.py
- Model Layer: models/cnn_model.py models/rnn_model.py
- Business Logic Layer: inference.py, debug_model.py
- Application Layer: web_app/app.py

4.3.3 Code-Level Explanation

FeatureExtractor (mediapipe_extractor.py) :

* Uses MediaPipe to extract pose landmarks
* Converts them into numerical feature vectors
* Currently merges all features into a single representation
* Issue: no separation between spatial and temporal features

Inference (inference.py) :

* Loads CNN, RNN, and LSTM models
* Runs predictions using the same input features
* Aggregates outputs into a final prediction
* Issue: no multi-stream processing or fusion mechanism

4.4 Design Rationale

Why to choose the architecture and design?

- Layered architecture ensures clarity and maintainability
- Presentation Layer: UI and user interaction
- Processing Layer: detection, logic, evaluation
- Data Layer: metrics storage and reporting
- Separation enables easier debugging and upgrades
- Modular design supports flexibility and reuse
- Easy maintenance leads to a scalable system.

Alternatives Acknowledgment

	Alternatives	Selected	Justification
Frontend	Flutter, React, etc.	Flask	Web-based UI, system compatibility and API handling
Processing	NumPy & Pandas	Both	Data processing enabling real-time performance and low computational cost
Models	Keras, gru, lstm, etc.	CNN & RNN	Highest accuracy and performance compared to 7 other models
Feedback	Direct classification, technical based	Structured Reports	Enabled scoring and metrics to prove performance instead of direct classification

Table 4: Alternatives Acknowledgment

4.5 Technical Background

1. **MediaPipe:**

   (a) Tracking Engine: MediaPipe serves as the foundation, detecting up to 2 hands per frame and mapping facial topology.

(b) Recognition Method: Uses custom 3D Landmark Heuristics rather than a generic $1 Recognizer, extracting 21 specific coordinates per hand to verify gestures geometrically (e.g., checking if thumb_tip_y ¡ thumb_ip_y for a Thumb Up).

(c) Gesture Scoring Equation: Performance is calculated by combining gesture variety and accuracy: Score = min(Unique Gestures / 5, 1.0) * 0.4 + (Gesture Accuracy) * 0.6.

(d) Normalizing equation to improve consistency:

i. Normalized x = Actual x (pixel) / Image Width

ii. Normalized y = Actual y (pixel) / Image Height

(e) Used in the **Detection Layer**.

2. **1$ Recognizer**:

(a) Lightweight gesture classification.

(b) The gesture recognition algorithm compares gesture shapes using distance matching.

(c) Lower distance means higher similarity between gestures.

(d) Used in the **Classification Layer**

3. **Machine Learning & AI Ensemble:**

(a) Model Architecture: Built on TensorFlow, Keras, and PyCaret, actively managing 8 distinct models that reshape flat data into 3D sequences (samples, 1, features) to simulate timelines.

(b) Ensemble Integration: Combines individual model predictions using a weighted probability average (weighted_proba += weight * proba) and a mathematical Voting System (np.bincount(x.astype(int)).argmax()).

(c) Emotion Performance Metric: Quantifies acting quality mathematically based on the rule that more emotional variation equals better acting

(d) Score = (Emotion Std.Dev.* 0.5) + (Face Confidence* 0.3) + ((1-Mean) * 0.2)

4. **Infrastructure & Data Processing:**

(a) Web & Visualization: Uses Flask for the API backend (handling 500MB video uploads) and Matplotlib to dynamically generate radar and bar charts for the UI.

(b) Data Handling: Relies on Pandas to structure multi-frame tracking data (pd.DataFrame) and NumPy for array operations and spatial math.

(c) Mathematical Application: NumPy calculates the exact Euclidean distance between coordinates (e.g., hand and ear)

(d) Distance = np.sqrt((hand_x - ear_x)**2 + (hand_y - ear_y)**2)

4.6 Evaluation Methodology

1. Quantitative Evaluation: Metrics such as accuracy, f1 score, precision and recall to evaluate performance.
2. Qualitative Evaluation: Assessment for usability and user satisfaction, enabling feature specification.
3. Comparative baselines: Compared to 7 other AI & ML models to optimize best model.
4. Validity Threats: Small sample size, controlled environment and computational power. No experienced director used the system.
5. Threats Mitigation: Loaded controlled dataset size to manage computational power and increasing dataset size gradually.

4.7 System Stability and Risks

Stability:

- Uses multiple inputs (video, voice, gestures) $\rightarrow$ more reliable results
- Combines different models to improve accuracy
- Clear processing steps keep the system consistent

Risks:

- Depends on good video/audio quality
- May not perform the same for every person
- Needs high processing power
- Limited data can affect accuracy

5 Testing and Evaluation

5.1 Testing Strategy

The methods used for testing a system's reliability and usability include:

1. Functional Testing: authentication; report generation; data retrieval
2. Non-Functional Testing: load times; concurrent users; API response time.
3. Usability Testing: task completion rates; error counts; user satisfaction.
4. Performance Testing: system performance given the expected growth trends.

The expected outcome of deploying this system is to provide:

- Objective casting decisions supported by quantitative metrics related to subjective assessments
- Reduce processing time by allowing for automated analysis and thus permit quicker evaluation of prospective talent
- Scalable operations, which means that there will be capability to review hundreds of auditions at once
- Actionable feedback, with regard to the actor
- Historical analysis related to the actor's development.

The testing is being executed using the automated testing dependency called PlayWright where 16 tests are done with 15 of them passed.

5.2 System and Integration Testing

A1. Video Input						
TEST ID	**TEST CASE**	**PRE-CONDITION**	**INPUT**	**STEPS**	**EXPECTED OUTPUT**	**RESULT**
TC-01	**File upload**	**Chose action and entered actor data**	**Video file (mp4, mov)**	**Choose file**	**Accepts browsing**	**PASSED**
TC-02	**Frame Capture**	**Video uploaded**	**Video frames**	**Run system**	**Frames processed continuously**	**PASSED**
TC-03	**Invalid file type**	**Video chosen**	**Uploading image (.jpeg)**	**Wrong fie input**	**Error handled gracefully**	**PASSED**
TC-04	**File ready to be processed**	**Successful video upload**	**Click Start button**	**Start analysis**	**Load report**	**PASSED**

Table 5: Video Input Test Cases

A2. Emotion Detection				
TEST ID	**TEST CASE**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-05	**Happy face**	**Actor smiling**	**”Happy” detected**	**PASSED**
TC-06	**Sad acting**	**Actor sad face**	**”Sad” detected**	**PASSED**
TC-07	**Neutral**	**No face expressions**	**Neutral detected**	**PASSED**
TC-08	**Lighting change**	**Low brightness video**	**Still detects correctly**	**PASSED**

Table 6: Emotion Detection Test Cases

A3. Face and Eye Tracking				
TEST ID	**TEST CASE**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-09	**No face**	**No face visible**	**No crash, no detection**	**PASSED**
TC-10	**Eye Tracking**	**Frequent gazing**	**Eye scoring and analysis**	**PASSED**
TC-11	**No voice**	**Silent video**	**No crash, no detection**	**PASSED**
TC-12	**Multiple faces**	**Several actors in video**	**Only one face tracked**	**PASSED**

Table 7: Face and Eye Tracking Test Cases

A4. Gesture Recognition				
TEST ID	**TEST CASE**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-13	**Known gesture**	**Forehand paddle hit**	**Correct classification**	**PASSED**
TC-14	**Unknown gesture**	**Boxing gesture**	**Rejected or low confidence**	**PASSED**
TC-15	**Speed variation**	**Slow and fast motion**	**Still recognized**	**PASSED**
TC-16	**Noise input**	**Alot of unkown gestures and figures around actor**	**No false positives**	**PASSED**

Table 8: Gesture Recognition Test Cases

A5. Report Generation					
TEST ID	**TEST CASE**	**PRE-CONDITION**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-17	**Generate report**	**Form filled and analysis done**	**Start button**	**Output created**	**PASSED**
TC-18	**Printing report**	**Report generated**	**Print Report button**	**Printed locally**	**PASSED**
TC19	**Score display**	**Model ready engine**	**Display**	**Numeric scores shown**	**PASSED**
TC-20	**Performance classification**	**Variables calculation**	**Known video classification: Bad**	**Good, Moderate or Bad shown**	**PASSED**
TC-21	**Visualization**	**Widgets displayed**	**Metrics produced by model**	**Widgets render**	**PASSED**
TC-22	**New analysis restart**	**Previous report fully generated**	**New Analysis Button**	**Button accessible**	**PASSED**
TC-23	**Variables Classification**	**Pipeline variables displayed**	**Variable individual model result**	**Independent variable scoring**	**PASSED**
TC-24	**Recommendations**	**Full classification completed**	**Bad performance in variables**	**Improvement advices**	**PASSED**

Table 9: Report Generation Test Cases

B1. Performance Testing					
TEST ID	**TEST CASE**	**PRE-CONDITION**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-25	**Processing delay**	**Model loaded successfully, system initialized**	**Frames in video file to process**	**less than 5000 ms**	**PASSED**
TC-26	**CPU usage**	**No background heavy applications running**	**Real-time processing workload (normal usage scenario)**	**less than 80%**	**PASSED**
TC-27	**Memory usage**	**System freshly started, no memory leaks**	**Continuous operation for 10–15 minutes**	**Stable**	**PASSED**

Table 10: Performance Test Cases

B2. Usability Testing					
TEST ID	**TEST CASE**	**PRE-CONDITION**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-28	**First-time user**	**No prior usage data**	**New user opens system for first time**	**Can run without help**	**PASSED**
TC-29	**UI clarity**	**Interface fully loaded**	**User navigates through all buttons/features**	**Buttons understandable**	**PASSED**
TC-30	**Report readability**	**System generates output reports**	**Sample report with detection results**	**Easy to interpret**	**PASSED**
TC-31	**Error messages**	**Error handling module active**	**Invalid inputs (eg. wrong file type)**	**Clear**	**PASSED**

Table 11: Usability Test Cases

B3. Security Testing					
TEST ID	**TEST CASE**	**PRE-CONDITION**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-32	**Invalid file upload**	**File upload feature enabled**	**Invalid file (.jpeg)**	**Rejected**	**PASSED**
TC-33	**Large file**	**Upload system with size limit configured**	**File exceeding allowed size: 609 MB**	**Handled safely**	**PASSED**
TC-34	**Data privacy**	**System connected to storage**	**Sensitive user data**	**No external leaks**	**PASSED**
TC-35	**Path injection**	**Input file paths**	**Malicious input: ../../../../etc/passwd**	**Prevented**	**PASSED**

Table 12: Security Test Cases

B4. Reliability Testing					
TEST ID	**TEST CASE**	**PRE-CONDITION**	**INPUT**	**EXPECTED OUTPUT**	**RESULT**
TC-36	**Multiple runs**	**System stable after first execution**	**Run system 5 times**	**Stable**	**PASSED**
TC-37	**Restart system**	**System previously executed**	**Restart and reload model**	**Works again**	**PASSED**
TC-38	**Load testing**	**System supports concurrent operations**	**Multiple users at once**	**User loads processed**	**FAILED**
TC-39	**Feature testing**	**All modules implemented**	**Test each feature independently**	**Individual functions working**	**PASSED**
TC-40	**Endurance testing**	**System stable under normal load**	**Running for 5 hours**	**Stable**	**PASSED**
TC-41	**Mean Time To Repair**	**Failure scenario simulated**	**Force crash and measure recovery time**	**less than 3s**	**PASSED**

Table 13: Reliability Test Cases

5.3 User Acceptance Testing

- Task Completion Rate: 100% of users completed tasks
- Time to Complete Tasks: about 3 minutes
- Error count (UI misunderstood): 2 mistakes
- User Satisfaction:
  - Using SUS (System Usability Scale)
  - Scoring Range: 0 - 100
  - Passing Score: greater than or equal 68
  - SUS Score of System: 85

5.4 Field Study

Implementation

- Upload video of performing scenes naturally.
- Use the system for real auditions

Data Collection

- Observations and Interviews
- Videos uploaded
- User feedback

Findings

- Backend processing time, report accuracy
- Actor feels stressed = gaze instability

5.5 System Performance Testing

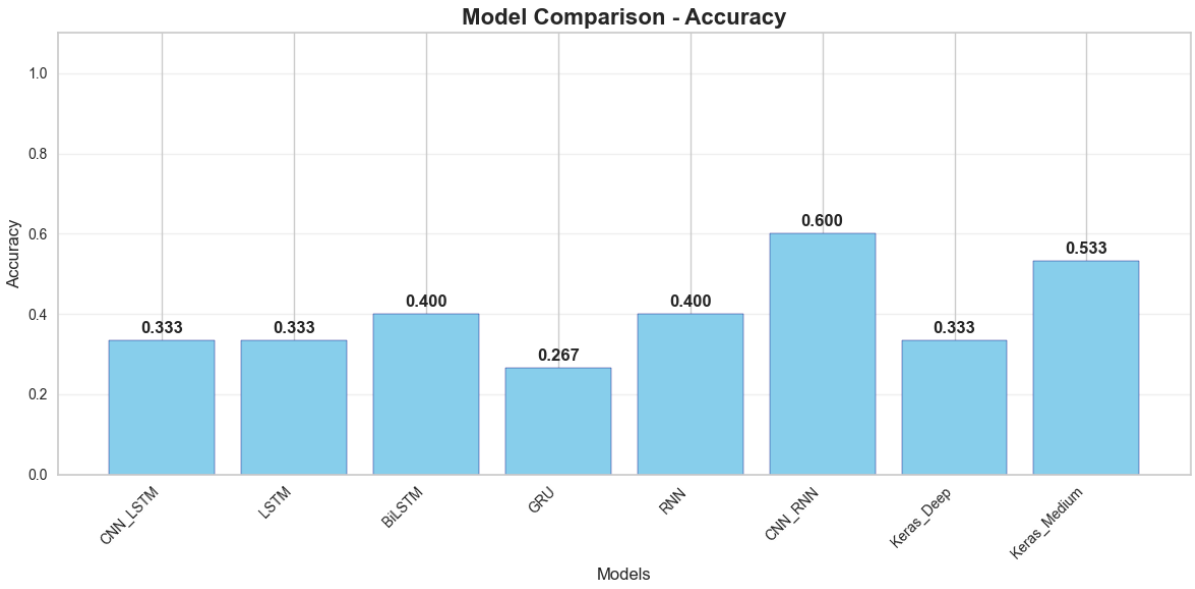

Figure 13: Model Accuracy Comparison Chart

	Model	Accuracy	Precision	Recall	F1-Score
5	CNN_RNN	0.600000	0.588889	0.600000	0.590572
7	Keras_Medium	0.533333	0.522222	0.533333	0.523906
2	BiLSTM	0.400000	0.400000	0.400000	0.400000
4	RNN	0.400000	0.291667	0.400000	0.335664
0	CNN_LSTM	0.333333	0.253968	0.333333	0.287879
1	LSTM	0.333333	0.151515	0.333333	0.208333
6	Keras_Deep	0.333333	0.277778	0.333333	0.252101
3	GRU	0.266667	0.317460	0.266667	0.266955

Figure 14: Model Metrics Comparison Figure

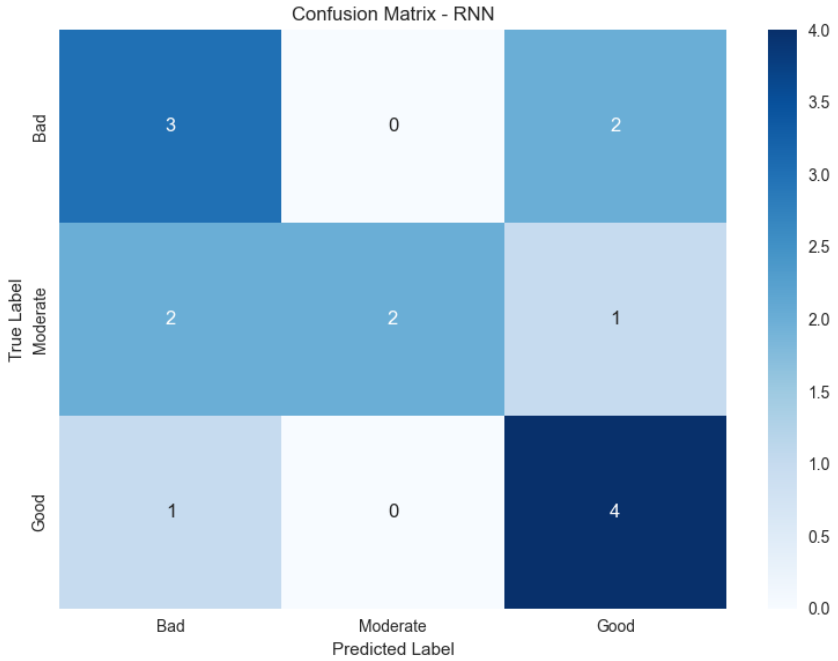

Figure 15: Models Confusion Matrix

6 Results and Conclusions

The implementation of AuditionSense proved how different AI analysis modules can work together in one pipeline. The result is as follows:

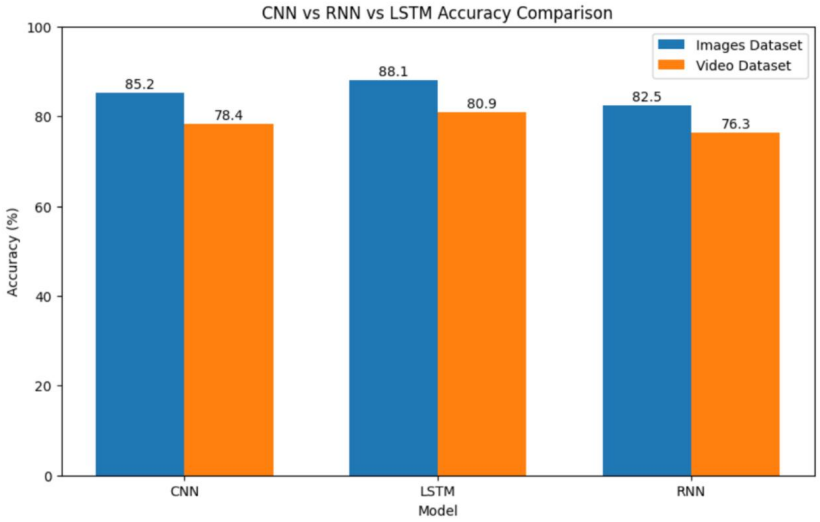

Figure 16: Accuracy Enhanced Results

6.1 Results

We were able to successfully:

- Upload and process multimedia files
- Detect emotions and gestures accurately
- Track poses and eye movements effectively
- Extract and analyze voice features
- Generate visual analytics and reports

The system processed user inputs to provide structured performance insights quickly and efficiently.

6.2 Performance

System performance with a modular architecture has improved by providing:

- Maintainability of analytical components
- Scalability of the analytical component
- Reuse of the analytical components

The parallel execution of the analysis modules has improved processing efficiency.

6.3 Conclusion

AuditionSense combines computer vision and audio technology to create a smart and scalable platform for multi-modal human performance analysis.

This project has demonstrated that integrating multiple AI technologies into a layered architecture will allow an analysis process that reduces manual efforts and increases depth of analysis. It will serve as a basis for future development of behavior analytics and human-computer interaction systems.

7 Lessons Learned & Problems Faced

7.1 Lessons Learned

- Benefits of a Modular Design: Having modules and components for each distinct part within a given system allows for easier debugging, testing and expansion of an overall project.
- Layered Architecture: Organizing system components using layers enhances communications and overall organization.
- Integration Challenges With Multiple AIs: The integration of multiple AI models has demonstrated the need for synchronized and standardized data flows.
- Media Processing Is Complicated: Combining audio and video files requires that thorough pre-processing and optimization be performed before the files are combined.
- The Importance of Data Quality: Analyses that are accurate and dependable are based on accurate and reliable data; examples of data quality issues include:
  - The level of lighting in the environment;
  - The clarity/quality of audio,
  - The quality of video.

7.2 Problem Faced

- Python as the primary programming language. Preferred libraries include: OpenCV, MediaPipe, dlib, DeepFace/FER, PyCaret, Flask, NumPy, pandas, matplotlib.
- To train an action segmentation algorithm, all training segments must be correctly labeled for a supervised training paradigm. Weakly-supervised or weakly-supervised/timestamp methods could be used as alternatives if budget constraints exist for labeling datasets.
- Managing video, audio, and gestures together.
- Making the system fast and smooth.
- Limited and complex dataset availability.
- Choosing the best model to accurately analyze the frames.

8 References and Appendices

- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems seventh edition.
- Evaluating Actors' Performance using Machine Learning for Gesture Classification. (2024)
- Improving Tele-Rehabilitation Performance: The Role of Ensemble Learning in Human Activity Recognition. (2025)
- GeeksforGeeks (2026) *Convolutional Neural Network (CNN) in Deep learning*, *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/
- Huijun Hu Jiawen Hao. "Beyond the Coach: Exploring the Efficacy of a Machine Learning Application for Improving Tennis Players' Performance". In: Aircconline.com 42.4 (2023). URL: https://aircconline.com/csit/papers/vol13/csit130907.pdf.
- Ksenia Kirillova, Xinran Lehto, and Liping Cai. "What triggers transformative tourism experiences?" In: Tourism Recreation Research 42.4 (2017), pp. 498–511. URL: https://doi.org/10.1080/02508281.2017.1342349.
- Preksha Pareek and Ankit Thakkar. A survey on video-based human action recognition: Recent updates, datasets, challenges, and Applications - Artificial Intelligence Review. Sept. 2020. URL: https://link.springer.com/article/10.1007/s10462-020 -09904-8#citeas
- Takashi Jindo et al. "Detection of similarities and differences within the same shot movement using artificial intelligence-based performance analysis: An example of a tennis service". In: International Journal of Racket Sports Science 5.1 (June 2023), pp. 34–46. URL: https://revistaseug.ugr.es/index.php/IJRSS/article/view/33247.
- Zijian Kuang and Xinran Tie. "IARG: Improved Actor Relation Graph Based Group Activity Recognition". In: Smart Multimedia. Ed. by Stefano Berretti and GuanMing Su. Cham: Springer International Publishing, 2022, pp. 29–40. ISBN: 978-3-031-22061-6.